

Principles of Programming Languages

Small-step Structural Operational Semantics

Andrei Arusoaie¹

¹Department of Computer Science

Outline

Small-step SOS

- Configurations

- Arithmetic expressions

- Boolean expressions

- Statements

Structural Operational Semantics

A language designer should understand the existing design approaches:

- ▶ Big-step structural operational semantics (discussed last week)
- ▶ Small-step structural operational semantics
- ▶ Denotational Semantics
- ▶ Modular operational semantics
- ▶ Reduction semantics with evaluation contexts
- ▶ Abstract Machines, the chemical abstract machine
- ▶ Axiomatic semantics

Structural Operational Semantics (SOS)

- ▶ SOS was proposed by Plotkin in 1981
- ▶ Simple framework to describe the behaviour of the language constructs as inference rules
- ▶ Rules specify transitions between program configurations
- ▶ Configurations are tuples of various kinds of data structures (e.g., trees, sets, lists)
 - ▶ capture various components of program states (environment, memory, stacks, registers, etc.)

Small-step Structural Operational Semantics

- ▶ A variant of SOS that captures the notion of one computational step
- ▶ A.k.a. *reduction semantics*, *transition semantics*, *one-step operational semantics*
- ▶ Small-step SOS for a PL: a set of small-step inference rules

Small-step Structural Operational Semantics

- ▶ A variant of SOS that captures the notion of one computational step
- ▶ A.k.a. *reduction semantics*, *transition semantics*, *one-step operational semantics*
- ▶ Small-step SOS for a PL: a set of small-step inference rules
- ▶ The transitions between small-step *configurations* is denoted by a simple arrow “ $\rightarrow$ ”: it captures only *one* computation step at a time, while the big-step transition denoted by $\Downarrow$ captures *all* computations in a single transition

Why do we need Small-step SOS?

- ▶ Big-step SOS issues:
 - ▶ non-deterministic behaviour

```
int main() {  
    int x = 1;  
    return (x = 1) + (x = 2);  
}
```

Why do we need Small-step SOS?

- ▶ Big-step SOS issues:

- ▶ non-deterministic behaviour

```
int main() {  
    int x = 1;  
    return (x = 1) + (x = 2);  
}
```

- ▶ cannot observe intermediate states (e.g., interleaving in concurrent languages)

Why do we need Small-step SOS?

- ▶ Big-step SOS issues:

- ▶ non-deterministic behaviour

```
int main() {  
    int x = 1;  
    return (x = 1) + (x = 2);  
}
```

- ▶ cannot observe intermediate states (e.g., interleaving in concurrent languages)
 - ▶ cannot naturally define the evaluation of expressions like `2 + nil` in languages where nats and lists can be mixed

Small-step SOS

Difference w.r.t. Big-step SOS:

Small-step SOS

Difference w.r.t. Big-step SOS:

- ▶ A small-step SOS transition encodes **only one** step

Small-step SOS

Difference w.r.t. Big-step SOS:

- ▶ A small-step SOS transition encodes **only one** step
- ▶ A big-step SOS transition can be simulated using multiple small-steps (transitive closure)

Small-step SOS

Difference w.r.t. Big-step SOS:

- ▶ A small-step SOS transition encodes **only one** step
- ▶ A big-step SOS transition can be simulated using multiple small-steps (transitive closure)
- ▶ Since big-step defines all computation steps in one transition it cannot capture non-determinism by default

Small-step SOS

Difference w.r.t. Big-step SOS:

- ▶ A small-step SOS transition encodes **only one** step
- ▶ A big-step SOS transition can be simulated using multiple small-steps (transitive closure)
- ▶ Since big-step defines all computation steps in one transition it cannot capture non-determinism by default
- ▶ Direct control over what to execute and when

Small-step SOS

Difference w.r.t. Big-step SOS:

- ▶ A small-step SOS transition encodes **only one** step
- ▶ A big-step SOS transition can be simulated using multiple small-steps (transitive closure)
- ▶ Since big-step defines all computation steps in one transition it cannot capture non-determinism by default
- ▶ Direct control over what to execute and when
- ▶ Therefore, small-step SOS is finer-grain than big-step SOS

The language that we use to illustrate SOS is IMP. It includes:

- ▶ Arithmetic expressions
- ▶ Boolean Expressions
- ▶ Statements: assignments, (possibly empty) blocks/sequences of statements, decisionals (if-then-else), loops (while)

Outline

Small-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Configurations

The small-step SOS configurations for IMP are a subset of the big-step SOS configurations:

- ▶ $\langle A, E \rangle$, where A is of type `AExp` and E is of type `Env`;
- ▶ $\langle B, E \rangle$, where B is of type `BExp` and E is of type `Env`;
- ▶ $\langle S, E \rangle$, where S is of type `Stmt` and E is of type `Env`.

Sequents and rules

- ▶ The **small-step SOS sequents** are binary relations over configurations: $C \rightarrow C'$
- ▶ $C \rightarrow C'$: the configuration C' is obtained from C after **one** step of computation

Sequents and rules

- ▶ The **small-step SOS sequents** are binary relations over configurations: $C \rightarrow C'$
- ▶ $C \rightarrow C'$: the configuration C' is obtained from C after **one** step of computation
- ▶ **Small-step SOS rules** have the following general form:

$$\frac{C_1 \rightarrow C'_1 \quad C_2 \rightarrow C'_2 \quad \cdots \quad C_n \rightarrow C'_n}{C_0 \rightarrow C'_0} \text{ cond.}$$

Outline

Small-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Small-step rules for IMP – 1: arithmetic expressions

Lookup and Addition:

SMALLSTEP-LOOKUP: $\langle x, \sigma \rangle \rightarrow \langle \sigma(x), \sigma \rangle$ if $\sigma(x) \neq \perp$

SMALLSTEP-ADD1:
$$\frac{\langle a_1, \sigma \rangle \rightarrow \langle a'_1, \sigma \rangle}{\langle a_1 +' a_2, \sigma \rangle \rightarrow \langle a'_1 +' a_2, \sigma \rangle}$$

SMALLSTEP-ADD2:
$$\frac{\langle a_2, \sigma \rangle \rightarrow \langle a'_2, \sigma \rangle}{\langle a_1 +' a_2, \sigma \rangle \rightarrow \langle a_1 +' a'_2, \sigma \rangle}$$

SMALLSTEP-ADD: $\langle i_1 +' i_2, \sigma \rangle \rightarrow \langle i_1 +_{nat} i_2, \sigma \rangle$

Note: + is non-deterministic!

Small-step rules for IMP – 1: arithmetic expressions

Multiplication:

$$\text{SMALLSTEP-MUL1: } \frac{\langle a_1, \sigma \rangle \rightarrow \langle a'_1, \sigma \rangle}{\langle a_1 *' a_2, \sigma \rangle \rightarrow \langle a'_1 *' a_2, \sigma \rangle}$$

$$\text{SMALLSTEP-MUL2: } \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'_2, \sigma \rangle}{\langle a_1 *' a_2, \sigma \rangle \rightarrow \langle a_1 *' a'_2, \sigma \rangle}$$

$$\text{SMALLSTEP-MUL: } \langle i_1 *' i_2, \sigma \rangle \rightarrow \langle i_1 *_{nat} i_2, \sigma \rangle$$

Small-step SOS

There are some noticeable differences w.r.t. big-step SOS:

- ▶ The lookup rule:
Small-step: $\langle x, \sigma \rangle \rightarrow \langle \sigma(x), \sigma \rangle$ vs. Big-step: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$;
- ▶ There is no rule for evaluating constants;
- ▶ More rules for evaluating addition/multiplication of arithmetic expressions!

Small-step SOS

There are some noticeable differences w.r.t. big-step SOS:

- ▶ The lookup rule:
Small-step: $\langle x, \sigma \rangle \rightarrow \langle \sigma(x), \sigma \rangle$ vs. Big-step: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$;
- ▶ There is no rule for evaluating constants;
- ▶ More rules for evaluating addition/multiplication of arithmetic expressions!
 - ▶ The reason is that both operands of an addition/multiplication can be evaluated using the one step relation $\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle$.

Small-step SOS

There are some noticeable differences w.r.t. big-step SOS:

- ▶ The lookup rule:
Small-step: $\langle x, \sigma \rangle \rightarrow \langle \sigma(x), \sigma \rangle$ vs. Big-step: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$;
- ▶ There is no rule for evaluating constants;
- ▶ More rules for evaluating addition/multiplication of arithmetic expressions!
 - ▶ The reason is that both operands of an addition/multiplication can be evaluated using the one step relation $\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle$.
 - ▶ Therefore, one can evaluate either the first argument or the second argument $\rightarrow$ two rules - one for each operand

Small-step SOS

There are some noticeable differences w.r.t. big-step SOS:

- ▶ The lookup rule:
Small-step: $\langle x, \sigma \rangle \rightarrow \langle \sigma(x), \sigma \rangle$ vs. Big-step: $\langle x, \sigma \rangle \Downarrow \langle \sigma(x) \rangle$;
- ▶ There is no rule for evaluating constants;
- ▶ More rules for evaluating addition/multiplication of arithmetic expressions!
 - ▶ The reason is that both operands of an addition/multiplication can be evaluated using the one step relation $\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle$.
 - ▶ Therefore, one can evaluate either the first argument or the second argument $\rightarrow$ two rules - one for each operand
 - ▶ Moreover, when both operands are completely evaluated we use an additional rule to obtain the result.

Simple derivation

- ▶ Let us consider $\sigma(x) = 10$. Here is a derivation for sequent $\langle 2 +' x, \sigma \rangle \rightarrow \langle 2 +' 10, \sigma \rangle$:

$$\frac{\overline{\langle x, \sigma \rangle \rightarrow \langle 10, \sigma \rangle} \text{ SMALLSTEP-LOOKUP}}{\langle 2 +' x, \sigma \rangle \rightarrow \langle 2 +' 10, \sigma \rangle} \text{ SMALLSTEP-ADD2}$$

- ▶ What about $\langle 2 +' x, \sigma \rangle \rightarrow \langle 12, \sigma \rangle$?

Sequences

- Reflexive and transitive closure $\rightarrow^*$:

$$\frac{\cdot}{a \rightarrow^* a} \text{ REFL}$$

$$\frac{a_1 \rightarrow a_2 \quad a_2 \rightarrow^* a_3}{a_1 \rightarrow^* a_3} \text{ TRAN}$$

Derivation example

- We can now prove $\langle 2 +' x, \sigma \rangle \rightarrow^* \langle 12, \sigma \rangle$:

$$\frac{\frac{\frac{}{\langle x, \sigma \rangle \rightarrow \langle 10, \sigma \rangle} \text{LOOKUP}}{\langle 2 +' x, \sigma \rangle \rightarrow \langle 2 +' 10, \sigma \rangle} \text{ADD2} \quad \frac{\frac{\frac{}{\langle 2 +' 10, \sigma \rangle \rightarrow \langle 12, \sigma \rangle} \text{ADD}}{\langle 2 +' 10, \sigma \rangle \rightarrow^* \langle 12, \sigma \rangle} \text{REFL} \quad \frac{}{\langle 12, \sigma \rangle \rightarrow^* \langle 12, \sigma \rangle} \text{TRAN}}{\langle 2 +' x, \sigma \rangle \rightarrow^* \langle 12, \sigma \rangle} \text{TRAN}$$

Outline

Small-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Small-step rules for IMP – 2: boolean expressions

The small-step SOS rules for boolean expressions are different from the big-step SOS rules:

- ▶ no rules for evaluating the boolean constants
- ▶ 3 rules for negation: one rule to evaluate of the negated boolean, two rules to evaluate the base cases;
- ▶ 3 rules for conjunction:
 - ▶ one which evaluates the first argument
 - ▶ one rule which evaluates the conjunction when the first argument is false (shortcircuited conjunction)
 - ▶ one rule which evaluate the conjunction when the first argument is true
- ▶ comparisons: each argument can be evaluated by a separate rule and there is one rule which compares two *concrete values*.

Small-step rules for IMP – 2: boolean expressions

Negations and conjunction:

$$\text{SMALLSTEP-NOT: } \frac{\langle b, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle !b, \sigma \rangle \rightarrow \langle !b', \sigma \rangle}$$

$$\text{SMALLSTEP-NOTTRUE: } \langle !\text{true}, \sigma \rangle \rightarrow \langle \text{false}, \sigma \rangle$$

$$\text{SMALLSTEP-NOTFALSE: } \langle !\text{false}, \sigma \rangle \rightarrow \langle \text{true}, \sigma \rangle$$

Small-step rules for IMP – 2: boolean expressions

Negations and conjunction:

$$\text{SMALLSTEP-NOT:} \quad \frac{\langle b, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle !b, \sigma \rangle \rightarrow \langle !b', \sigma \rangle}$$

$$\text{SMALLSTEP-NOTTRUE:} \quad \langle !\text{true}, \sigma \rangle \rightarrow \langle \text{false}, \sigma \rangle$$

$$\text{SMALLSTEP-NOTFALSE:} \quad \langle !\text{false}, \sigma \rangle \rightarrow \langle \text{true}, \sigma \rangle$$

$$\text{SMALLSTEP-AND1:} \quad \frac{\langle b_1, \sigma \rangle \rightarrow \langle b'_1, \sigma \rangle}{\langle b_1 \text{ and' } b_2, \sigma \rangle \rightarrow \langle b'_1 \text{ and' } b_2, \sigma \rangle}$$

$$\text{SMALLSTEP-ANDFALSE:} \quad \langle \text{false and' } b_2, \sigma \rangle \rightarrow \langle \text{false}, \sigma \rangle$$

$$\text{SMALLSTEP-ANDTRUE:} \quad \langle \text{true and' } b_2, \sigma \rangle \rightarrow \langle b_2, \sigma \rangle$$

Small-step rules for IMP – 2: boolean expressions

Comparisons:

$$\text{SMALLSTEP-LT1: } \frac{\langle a_1, \sigma \rangle \rightarrow \langle a'_1, \sigma \rangle}{\langle a_1 <' a_2, \sigma \rangle \rightarrow \langle a'_1 <' a_2, \sigma \rangle}$$

$$\text{SMALLSTEP-LT2: } \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'_2, \sigma \rangle}{\langle i_1 <' a_2, \sigma \rangle \rightarrow \langle i_1 <' a'_2, \sigma \rangle}$$

$$\text{SMALLSTEP-LT: } \langle i_1 <' i_2, \sigma \rangle \rightarrow \langle i_1 <_{nat} i_2, \sigma \rangle$$

Small-step rules for IMP – 2: boolean expressions

Comparisons:

$$\text{SMALLSTEP-LT1: } \frac{\langle a_1, \sigma \rangle \rightarrow \langle a'_1, \sigma \rangle}{\langle a_1 <' a_2, \sigma \rangle \rightarrow \langle a'_1 <' a_2, \sigma \rangle}$$

$$\text{SMALLSTEP-LT2: } \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'_2, \sigma \rangle}{\langle i_1 <' a_2, \sigma \rangle \rightarrow \langle i_1 <' a'_2, \sigma \rangle}$$

$$\text{SMALLSTEP-LT: } \langle i_1 <' i_2, \sigma \rangle \rightarrow \langle i_1 <_{nat} i_2, \sigma \rangle$$

$$\text{SMALLSTEP-GT1: } \frac{\langle a_1, \sigma \rangle \rightarrow \langle a'_1, \sigma \rangle}{\langle a_1 >' a_2, \sigma \rangle \rightarrow \langle a'_1 >' a_2, \sigma \rangle}$$

$$\text{SMALLSTEP-GT2: } \frac{\langle a_2, \sigma \rangle \rightarrow \langle a'_2, \sigma \rangle}{\langle a_1 >' a_2, \sigma \rangle \rightarrow \langle a_1 >' a'_2, \sigma \rangle}$$

$$\text{SMALLSTEP-GT: } \langle i_1 >' i_2, \sigma \rangle \rightarrow \langle i_1 >_{nat} i_2, \sigma \rangle$$

Derivation example

$$\frac{\frac{\langle 1 + ' 3, \sigma \rangle \rightarrow \langle 4, \sigma \rangle}{\langle 1 + ' 3 < ' 5, \sigma \rangle \rightarrow \langle 4 < ' 5, \sigma \rangle} \text{ADD} \quad \frac{\frac{\langle 4 < ' 5, \sigma \rangle \rightarrow \langle true, \sigma \rangle}{\langle 4 < ' 5, \sigma \rangle \rightarrow^* \langle true, \sigma \rangle} \text{LT} \quad \frac{\langle true, \sigma \rangle \rightarrow^* \langle true, \sigma \rangle}{\langle 4 < ' 5, \sigma \rangle \rightarrow^* \langle true, \sigma \rangle} \text{REFL}}{\langle 1 + ' 3 < ' 5, \sigma \rangle \rightarrow^* \langle true, \sigma \rangle} \text{TRAN}$$

Outline

Small-step SOS

Configurations

Arithmetic expressions

Boolean expressions

Statements

Small-step rules for IMP – 3: statements

Assignments and Sequences:

$$\text{SMALLSTEP-ASSIGN-2: } \frac{\langle a, \sigma \rangle \rightarrow \langle a', \sigma \rangle}{\langle x ::= a, \sigma \rangle \rightarrow \langle x ::= a', \sigma \rangle}$$

$$\text{SMALLSTEP-ASSIGN: } \langle x ::= i, \sigma \rangle \rightarrow \langle \text{skip}, \sigma[i/x] \rangle$$

$$\text{SMALLSTEP-SEQ: } \frac{\langle s_1, \sigma \rangle \rightarrow \langle s'_1, \sigma' \rangle}{\langle s_1 ;; s_2, \sigma \rangle \rightarrow \langle s'_1 ;; s_2, \sigma' \rangle}$$

$$\text{SMALLSTEP-SKIP: } \langle \text{skip} ;; s_2, \sigma \rangle \rightarrow \langle s_2, \sigma \rangle$$

Small-step rules for IMP – 3: statements

Decisionals and loops:

$$\text{SMALLSTEP-ITE:} \quad \frac{\langle b, \sigma \rangle \rightarrow \langle b', \sigma \rangle}{\langle \text{ite } b \ s_1 \ s_2, \sigma \rangle \rightarrow \langle \text{ite } b' \ s_1 \ s_2, \sigma \rangle}$$

$$\text{SMALLSTEP-ITETRUE:} \quad \langle \text{ite } b_{\text{true}} \ s_1 \ s_2, \sigma \rangle \rightarrow \langle s_1, \sigma \rangle$$

$$\text{SMALLSTEP-ITEFALSE:} \quad \langle \text{ite } b_{\text{false}} \ s_1 \ s_2, \sigma \rangle \rightarrow \langle s_2, \sigma \rangle$$

$$\text{SMALLSTEP-WHILE:} \quad \langle \text{while } b \ s, \sigma \rangle \rightarrow \langle \text{ite } b(s; ; \text{while } b \ s) \text{ skip}, \sigma \rangle$$

Derivation example

We show here a derivation example for the sequent

$$\langle x := n, \sigma_1 \rangle \rightarrow \langle \text{skip}, \sigma_2 \rangle,$$

where $\sigma_1(n) = 10$ and $\sigma_2 = \sigma_1[10/x]$ (with rule labels):

$$\frac{\frac{\frac{}{\langle n, \sigma_1 \rangle \rightarrow \langle 10, \sigma_1 \rangle} \text{LOOKUP} \quad \frac{\frac{}{\langle x := 10, \sigma_1 \rangle \rightarrow \langle \text{skip}, \sigma_2 \rangle} \text{ASSIGN} \quad \frac{}{\langle \text{skip}, \sigma_2 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle} \text{REFL}}{\langle x := 10, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle} \text{TRAN} \quad \frac{\langle x := n, \sigma_1 \rangle \rightarrow \langle x := 10, \sigma_1 \rangle \quad \langle x := 10, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle}{\langle x := n, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle} \text{TRAN}}{\langle x := n, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle} \text{TRAN}$$

Without labels:

$$\frac{\frac{\frac{}{\langle n, \sigma_1 \rangle \rightarrow \langle 10, \sigma_1 \rangle}}{\langle x := n, \sigma_1 \rangle \rightarrow \langle x := 10, \sigma_1 \rangle} \quad \frac{\frac{\frac{}{\langle x := 10, \sigma_1 \rangle \rightarrow \langle \text{skip}, \sigma_2 \rangle} \quad \frac{}{\langle \text{skip}, \sigma_2 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle}}{\langle x := 10, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle}}{\langle x := n, \sigma_1 \rangle \rightarrow^* \langle \text{skip}, \sigma_2 \rangle}$$

Bibliography

1. Chapter Small-Step Operational Semantics in **Software Foundations - Volume 2**, Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, Brent Yorgey
<https://softwarefoundations.cis.upenn.edu/plf-current/Smallstep.html>